and retrieved through a repository.

The main elements in the interaction shown in figure 5.3 are database drivers, entities, and repositories:

- *Database drivers*—Components that provide integration with a specific database (through *connection factories*). For relational databases, you can use a JDBC driver (Java Database Connectivity API) in imperative/blocking applications or an R2DBC driver in reactive/non-blocking applications. For non-relational databases, each vendor has its own dedicated solution.
- *Entities*—Domain objects that are persisted in a database. They must contain a field to uniquely identify each instance (a *primary key*) and can use dedicated annotations to configure the mapping between Java objects and database entries.
- *Repositories*—The abstractions used for data storage and retrieval. Spring Data provides basic implementations, which are further extended by each module to provide features specific to the database in use.

This section will show you how to use Spring Data JDBC to add data persistence to a Spring Boot application like Catalog Service. You'll configure a connection pool to interact with a PostgreSQL database through the JDBC driver, define entities to be persisted, use repositories to access data, and work with transactions. Figure 5.4 shows how the Polar Bookshop architecture will look by the end of this chapter.